Object-Oriented Programming

Lab 2 — Classes and Objects

What is an object, really?

Total Marks: 20
Submission: GitHub Repository

Where we are. In Lab 1 you built a student record system three times and ended up with a class. You saw that a class lets you keep data and the functions that work on it in one place, and that it protects the data from being changed by accident. Today you will slow down and look at the ideas behind that, starting from a completely different real-world situation.

Problem 1 Modelling a Bank Account [16 marks]

The situation

A bank needs software to manage customer accounts. Each account belongs to one person and has a current balance. Customers can deposit money, withdraw money (but never go below zero), and check their balance. Multiple customers have accounts at the same time, and each one is completely independent of the others.

Part A — Design before you code [4 mark]

Before writing a single line of code, answer these four questions in a comment block at the top of your file.

a. What information does every account need to store? List each piece and the type you would use.
b. What are the three actions a customer can perform on their account? For each one, write down what it needs to know (inputs) and what it produces (output or change).
c. A colleague suggests storing all account balances in one big array in `main`, with a separate array for owner names, just like Task 1 of Lab 1. What specific problem would appear the moment you need to handle 200 customers?
d. Suppose the bank later wants to add a transaction limit per day. Where exactly would you add that — in `main`, or somewhere else? Why?

Part B — Write the class [8 marks]

Implement a class called `BankAccount` in Java or C#.

1. The owner's name and the balance must be stored as **private** fields.
2. The constructor takes the owner's name as a parameter and sets the starting balance to `0`.
3. Implement three public methods:

- `deposit(amount)` — adds `amount` to the balance. If `amount` is zero or negative, print an error and do nothing.
- `withdraw(amount)` — subtracts `amount` from the balance. If the result would go below zero, print `"Insufficient funds"` and do nothing.
- `printStatement()` — prints the owner's name and current balance.

4. Write a `main` method that creates **three separate** `BankAccount` objects for three different customers, performs a mix of deposits and withdrawals on each, and calls `printStatement()` on each.

> ▷ **Notice**
>
> Do not add any static variables or global state. Each `BankAccount` object must track its own balance entirely on its own.

Part C — Objects in memory [4 mark]

Add a comment block *after* your `main` method answering the following.

You wrote something like this in `main`:

```
BankAccount alice = new BankAccount("Alice");
BankAccount bob   = new BankAccount("Bob");
```

a. `BankAccount` is written once in your source file. `alice` and `bob` are two separate things created from it. What is the word for what `BankAccount` is, and what is the word for what `alice` and `bob` are?

b. When you call `alice.deposit(500)`, which balance changes — Alice's, Bob's, or both? How does the program know which one to update?

c. Draw a simple box diagram (in ASCII or words) showing what is in memory after both lines above have run. Each box should show the field names and their values.

d. If you wrote `alice = bob;` and then called `alice.deposit(100)`, what would happen to Bob's balance? Predict the result and explain why.

Problem 2 Stack and Queue from `StudentList` [4 marks]

Background

At the end of Lab 1 you were asked to think about this without implementing it:

> *What change to* ***`add`*** *and* ***`removeAt`*** *would turn* ***`StudentList`*** *into a* ***stack****? Into a* ***queue****?*

Today you will implement both, step by step. Both structures use the same fixed-size array you already have. The only thing that changes is *which end of the array* you add to and remove from.

> **The key idea.** A stack and a queue are not new kinds of data — they are the same array your `StudentList` already has, but with a rule about how you are allowed to use it. The class enforces that rule through its methods. Outside code cannot break the rule because it cannot touch the array directly. This is exactly why the array is `private`.

What a stack does

A stack works like a pile of papers on your desk. You always add a new sheet on *top*, and when you take one away you always take from the *top*. The last sheet you put down is the first one you pick up.

The two operations are:

- **push** — add a student at the top of the pile (the end of the array).
- **pop** — remove and return the student at the top of the pile (the last used position).

There is no shifting of elements. `count` just goes up by one on a push and down by one on a pop.

Part A — Implement `StudentStack` [2 marks]

Create a new class called `StudentStack`. Copy your `StudentList` private fields (the array and the count) into it as a starting point.

Implement these public methods:

Method	Behaviour
`push(name, mark)`	Add the student at position `count`. Print an error if the stack is full.
`pop()`	Remove and print the student at position `count - 1`. Print an error if the stack is empty.
`peek()`	Print the student at the top without removing them.
`display()`	Print all students from bottom to top (index 0 upward).

Write a `main` method that pushes five students, calls `peek()`, pops three times, and calls `display()`.

▷ **Notice**

`pop()` must not leave stale data sitting at a high index and confuse later operations. Decrementing `count` is sufficient — you do not need to zero out the removed slot.

What a queue does

A queue works like a line of people waiting. New people join at the *back* of the line, and the person at the *front* is served first. The first person to arrive is the first person to leave.

The two operations are:

- **enqueue** — add a student at the back (position `count`).
- **dequeue** — remove the student at the front (position `0`) and shift everyone else one step forward.

Notice that dequeue does require a shift, and it shifts toward index 0, not away from it. Compare this with `removeAt` in Lab 1 — the logic is the same, but it always operates at position 0.

Part B — Implement `StudentQueue` [1 marks]

Create a new class called `StudentQueue`. Again, copy the same private fields as your starting point.

Implement these public methods:

Method	Behaviour
`enqueue(name, mark)`	Add the student at position `count`. Print an error if the queue is full.
`dequeue()`	Remove the student at position 0, shift all others left by one, decrement `count`, and print the removed student. Print an error if the queue is empty.
`front()`	Print the student at position 0 without removing them.
`display()`	Print all students from front to back (index 0 onward).

Write a `main` method that enqueues five students, calls `front()`, dequeues three times, and calls `display()`.

Part C — Realization [1 marks]

Add a comment block at the top of each file answering the following questions. Two or three sentences per question is enough.

In `StudentStack.java` (or `.cs`):

a. `push` and `pop` never shift any elements. What does this mean for speed when the stack has 50 students versus 5 students?
b. Your `StudentStack` and `StudentQueue` both have a private field called `count` (or similar). If someone writes code that uses both classes at the same time, can one object's `count` interfere with the other's? Why not?

In `StudentQueue.java` (or `.cs`):

a. `dequeue` always shifts the entire remaining array one step to the left. If the queue holds 50 students and you dequeue 45 times, roughly how many individual element moves does that require in total?
b. You have now written three classes — `StudentList`, `StudentStack`, and `StudentQueue` — all using the same private array and count. The only real difference is which methods are available to outside code. In one sentence, explain why hiding the array as `private` makes this possible.

"A class is a description. An object is the thing itself.
You write the description once, but you can create as many things as you need."
